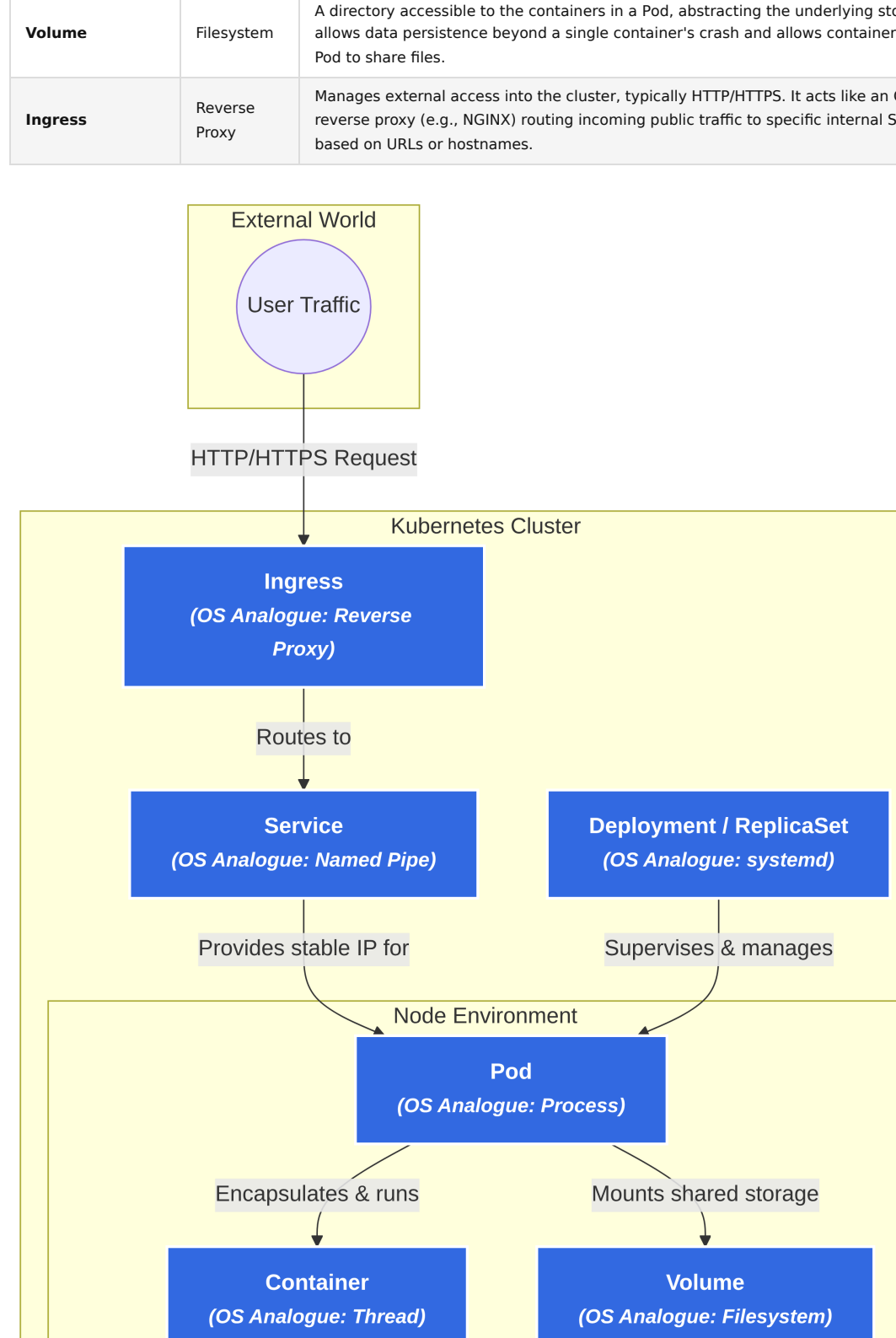
2 - First Cluster

In this lesson you will learn the basics of Kubernetes elements, and then you will start your first cluster.

Kubernetes elements

Kubernetes is like an Operating System for your cluster. Just as an OS manages processes, threads, files, and networking, Kubernetes manages the following elements:

Kubernetes element	OS analogue	Explanation
Container	Thread	The actual execution unit. While technically isolated processes under the hood, conceptually they act like threads: they share local network (localhost) and storage with other containers in the same Pod.
Pod	Process	The smallest deployable unit in Kubernetes. It encapsulates one or more containers (threads), providing a shared environment (namespaces) for them to run and communicate natively.
Deployment / ReplicaSet	`systemd`	Manages the lifecycle of Pods. It acts as a supervisor that ensures the desired number of replicas are always running, automatically restarting crashed Pods and managing rolling updates.
Service	Named Pipe	Provides a stable internal IP and DNS name to route traffic to a dynamic set of Pods. It shields other applications from the fact that backend Pods are constantly dying and changing IP addresses.
Volume	Filesystem	A directory accessible to the containers in a Pod, abstracting the underlying storage. It allows data persistence beyond a single container's crash and allows containers in a Pod to share files.
Ingress	Reverse Proxy	Manages external access into the cluster, typically HTTP/HTTPS. It acts like an OS-level reverse proxy (e.g., NGINX) routing incoming public traffic to specific internal Services based on URLs or hostnames.



Chassis-demo cluster

In the following steps, we will recreate the service described in chassis-demo docker-compose.yaml by using each of the elements described above.

1. First build and push your product-service container on Docker Hub

```
cd modules/chassis/chassis-java/labs/chassis-demo/product-service
mvn package
docker build . -t $DOCKER_USERNAME/product-service
docker push $DOCKER_USERNAME/product-service
```

> 📘 ***Info***
> *This is essential since having the container image in a remote repository is required to make it accessible to your cluster.*

2. Create the cluster

```
k3d cluster create first-cluster -s 1 -a 2
```

3. Create Postgres Deployment

```
kubectl create deployment postgres --image=postgres:17-alpine
kubectl set env deployment/postgres POSTGRES_USER=user POSTGRES_PASSWORD=secret POSTGRES_DB=jdbc_schema
```

> 📘 ***Info***
> *Postgres and the Product Service container will be managed in two separate Deployments. This is best practice since it is often necessary to scale the frontend and backend of a service differently. The Kubernetes Deployment object automatically defines a Pod and its associated containers.*

> ⚠️ ***Warning***
> *For now the database will be run without a Volume attached. This is critical in production environment since this makes the data ephimeral.*

5. Create a Service to make the database reachable from inside the cluster
   To create a service you can use `kubectl expose` command.

```
kubectl expose deployment postgres --port=5432
```

> 📘 ***Info****: Kubernetes DNS*
> *By exposing this deployment, Kubernetes creates an internal DNS entry named postgres. Any other application in the cluster can now reach the database simply by pointing to postgres:5432 — exactly like Docker Compose!*

6. Create `product-service` Deployment

```
kubectl create deployment product-service --image=$DOCKER_USERNAME/product-service --port=8080
kubectl set env deployment/product-service SPRING_PROFILES_ACTIVE=docker
SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/jdbc_schema
```

7. Create a Service for `product-service`

```
kubectl expose deployment product-service --port=8080
```

8. Provide an Ingress to the system

```
kubectl create ingress chassis-demo-ingress --rule="chassis.local/*=product-service:8080"
```

Get the Ingress IP with

```
kubectl get ingresses
```

> 📘 ***Info***
> *This will route any request aimed at the domain `chassis.local` to the `product-service` Service on port 8080.*

> 💡 ***Tip****: Local DNS Resolution*
> *To test this locally in your browser or via terminal, you will need to map chassis.local to your local machine. You can do this by adding `INGRESS_IP chassis.local` to your `/etc/hosts` file.*

You can verify that everything is working properly both from Headlamp and from kubectl command line:

```
# Verify both Pods are active
kubectl get pods

# Verify both Services exist and are routing ports
kubectl get svc

# Verify the external mapping is active
kubectl get ingress
```

As you might have noticed, using imperative commands has its limits. For instance, it becomes incredibly difficult to manage multi-container deployments and volumes this way, and the commands become long and hard to maintain very quickly.
A more robust and scalable way to manage a Kubernetes cluster is by writing declarative **YAML configuration files**.

Kubernetes YAML Structure

Before we dive into a full configuration file, it is essential to understand its basic anatomy. No matter how complex a Kubernetes object gets, almost every YAML file is built around four foundational root fields:

- `apiVersion` : Specifies which version of the Kubernetes API you are using to create the object (e.g., v1, apps/v1). Different resources live under different API groups.
- `kind` : Defines the type of resource you want to create, such as a Deployment, Service, or PersistentVolumeClaim.
- `metadata` : Contains data used to uniquely identify the object, primarily its name and any organizational labels.
- `spec` : Short for "specification," this is the heart of the file. It defines the desired state of the resource-like which container image to use, what ports to open, or how many replicas to run. Kubernetes will constantly monitor the cluster to ensure the actual state matches this spec.

Additionally, you will often see three dashes (`---`) used throughout these configurations. This is standard YAML syntax that separates multiple distinct documents within a single file. It allows us to bundle an entire application stack (databases, backend services, routing rules) into one deployable file.
As you might have noticed, using imperative commands has its limits. For instance, it becomes incredibly difficult to manage multi-container deployments and volumes this way, and the commands become long and hard to maintain very quickly.
A more robust and scalable way to manage a Kubernetes cluster is by writing declarative YAML configuration files.

For example, this single configuration file replaces all the individual commands we ran in the previous steps. You can save this as `chassis-demo.yaml` :

```yaml
---
# 1. PersistentVolumeClaim (The request for storage)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pg-data-pvc
spec:
  accessModes:
    - ReadWriteOnce # This means the volume can be mounted as read-write by a single node
  resources:
    requests:
      storage: 1Gi  # Request 1 Gigabyte of storage space

---
# 2. Postgres Deployment (Updated)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:17-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              value: user
            - name: POSTGRES_PASSWORD
              value: secret
            - name: POSTGRES_DB
              value: jdbc_schema
          volumeMounts:
            - name: pg-data-volume
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: pg-data-volume
          persistentVolumeClaim:
            claimName: pg-data-pvc

---
# 3. Postgres Service
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432

---
# 4. Product-Service Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          # ⚠️ Replace <DOCKER_USERNAME> with your actual Docker Hub username!
          image: <DOCKER_USERNAME>/product-service
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: docker
            - name: SPRING_DATASOURCE_URL
              value: jdbc:postgresql://postgres:5432/jdbc_schema

---
# 5. Product-Service Service
apiVersion: v1
kind: Service
metadata:
  name: product-service
spec:
  selector:
    app: product-service
  ports:
    - port: 8080
      targetPort: 8080

---
# 6. Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: chassis-demo-ingress
spec:
  rules:
    - host: chassis.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: product-service
                port:
                  number: 8080
```

After having created the YAML configuration file, you can apply it by running `kubectl apply` :

```
kubectl apply -f FILENAME.yaml
```

It is highly suggested to mix imperative and declarative approaches when building complex systems. You can use imperative commands to quickly generate a base YAML mockup (without actually deploying it) by using the `--dry-run=client` flag:

```
kubectl create deployment NAME --image=IMAGE --dry-run=client -o yaml > kubernetes.yaml
```

Then, you can open `kubernetes.yaml` in your editor and add details to it. If you aren't sure how a specific field should be formatted, you can inspect Kubernetes objects using the explain command:

```
kubectl explain OBJECT
```

For example, you can get every Pod YAML field by running:

```
kubectl explain pod
```